MIPRO 2014, 26-30 May 2014, Opatija, Croatia

Solving tourist trip planning problem via a Simulated Annealing Algorithm

Kadri Sylejmani*, Atdhe Muhaxhiri[x], Agni Dika* and Lule Ahmedi*
*Department of Computer Engineering, University of Prishtina, Prishtina, Kosova
[x]ProSolution, Prishtina, Kosova
kadri.sylejmani@uni-pr.edu, atdhe.muhaxhiri@prosolution-ks.com, agni.dika@uni-pr.edu, lule.ahmedi@uni-pr.edu

***Abstract*—Nowadays, by help of tourist trip planners, in many travel destinations, the trip itinerary could be prepared automatically. The trip planners enable customization of trip itinerary based on tourist preferences, available time and budget, where the itinerary is optimized by using optimization techniques from the field of metaheuristics. In this paper, we present a new approach based on simulated annealing algorithm for solving the tourist trip planning problem. The hard constraints include limited trip duration, working hours of points of interest (POIs) and tourist budget, while tourist preferences and traveling time comprise the soft constraints, which take part in a two component function for evaluation of quality of solutions. The search space is explored by using three types of operators, namely *insert*, *swap* and *shake*. The algorithm is tested against some real test data for the city of Prishtina in Kosova. In addition to fine tuning the values of algorithm parameters, further experiments are made for exploring different approaches of construction of initial solution, time duration of algorithm execution for various trip lengths, and influence of *shake* operator into the solution quality.**

I. Introduction

A tourist trip itinerary is plan for visiting a number of POIs in city (or a wider geographic region) during a limited period of time. In general, the itinerary contains a list of visits to POIs that are ordered based on the time of visit, where each visit might be annotated with a starting and ending time, visit duration, a short description of POI, etc. In addition, a trip itinerary might include a map with navigational information for traversing between individual POIs.

Usually, in a touristic city/region there might a high number of POIs which could be potentially visited, nevertheless the trip duration is usually limited to few days, and as result, only a subset of them can be actually visited. In addition, often it happens that the tourists preferences about types (e.g. monument, castle, statue, etc.) and categories (e.g. architecture, archeology, nature, etc.) of POIs are such that require visiting POIs that are located far from each other. Therefore, it is a complex task for the tourist to manually prepare the trip itinerary that would include the POIs that are the most preferred to her/him.

In general, the tourist trip planning problem is known as Tourist Trip Design Problem (TTDP) [1]. Given a number of POIs in a city/region, the goal is to prepare an itinerary that includes visits to POIs that meet, to a great extent, the preferences of the tourist. Depending on the kinds of features or constraints being considered for the problem, different problem modelings from the literature could be used to model the tourist trip planning problem. The Orienteering Problem (OP) [2] and the Team Orienteering Problem (TOP) [3] are used to model single and multiple day itineraries, respectively. The OP with Time Windows (OPTW) [4] and TOP with Time Windows (TOPTW) [5] enable modeling working hours of POIs, whereas Multi Constraint TOPTW (MCTOPTW) [6] allows enforcing additional constraints for the itinerary, such as maximum budget to spent, or maximum number of POIs of certain type (e.g. statue, castle, etc.) to be visited. The OP with Hotel Selection (OPHS) [7] enables selection of a hotel at the end of the trip of each tour.

Golden et al. [8] prove that OP is a NP hard problem, therefore many researcher tackle OP and its deriving extensions by using various approaches from the field of metaheuristics. Vansteenwegen et al. [9] discuss the most effective approaches that solve OP and its respective extensions.

In the process of trip itinerary planning, a method that estimates the interest of the tourist about POIs is required [10]. A such a method would consider the preferences of the tourist and features of the POIs, and based on their reciprocal match, it would generate the estimated satisfaction factors of the tourist with each POI. Nevertheless, introducing a new such a method is out of scope of this paper, hence, a method from the literature is used for that purpose.

The goal of this paper is to present a new algorithm that plans the tourist trip itinerary by considering multiple day tours, tourist budget, working hours of POIs and breaks during the trip of a day. In comparison to existing modelings in the literature for the trip planning problem, where optimization is done by only maximizing the satisfaction factor of a tourist with POIs, in our case, the optimization includes a second component, that is minimization of the total traveling time of the tourist.

The remaining of this paper is organized as follows: Section II discusses the state of the art algorithms, Section III presents the mathematical modeling of the problem, Section IV presents the proposed solution, Section V gives the computational experiments and Section VI outlines the main conclusions of the paper.

II. State of The Art

Vansteenwegen et al. [5] are the first to model TTDP as TOPTW problem. As result, they develop an approach based on the Iterated Local Search (ILS) framework. The neighborhood structure of the algorithm comprises of an insert step (for inserting new POIs) and a shake step (for escaping from local optima). The insert step adds consecutively new

points into the tours. The shake step removes one or more visits from each tour. The computation time of this algorithm is, with a factor several hundred, faster than the other presented approaches in the literature for TOPTW problem. This feature makes the algorithm suitable for application in the domain of tourist trip planning. Later, Garcia et al. [6] extend ILS to solve the Multi Constraint TOPTW, which allows modeling the constraints about maximum allowed budget to be spent and maximum allowed POIs of certain type or category to be visited.

Gavalas et al. [11] solve TOPTW by using two analogous approaches from cluster-based heuristics named as Cluster Search Cluster Ratio (CSCRatio) and Cluster Search Cluster Routes (CSCRoutes). The presented approaches aim at encouraging visits to dense areas that consist of a high density of "good" points (with high satisfaction factors). Both heuristics employ a clustering procedure to organize points into clusters based on distance from dense areas. This increases the probability that more visits would to take place inside individual clusters, and as result, two positive effects would be achieved, the decrease of duration of tours and minimization of number of transfers between different clusters. In terms of quality of solutions, this approach performs better than ILS [5], while the computation time remains in comparable level.

Garcia et al. [12] solve a variant of OPTW problem called Time Dependent Orienteering Problem with Time Windows (TDOPTW), which models the variability of traveling time between POIs. The TDOPTW is used to model different modes of traveling between any two POIs, such as walking, bicycle, public transportation or car. The ILS algorithm [5] is hybridized with Dijkstras Shortest Path algorithm to solve the TDOPTW problem. The Dijkstras algorithm is used to calculate the average travel times between all points available. Due to the constraint for a real time computation, the traveling times are calculated in an off-line-mode and saved for letter use by the ILS algorithm. The computation experiments performed with the data of city of San Sebastian in Spain, prove that the approach is able to generate valid routes in real time.

Souffriau et al. [13] extend MCTOPTW problem to the MCTOP with Multiple Time Windows (MCTOPMTW), where multiple working shifts of POIs can be modeled. The proposed approach for solving MCTOPMTW is a hybridization between ILS and Gready Randomized Adaptive Search Procedure (GRASP) algorithms. This approach is also based on the ILS approach of Vansteenwegen et al. [5] for TOPTW. The ILS-GRASP approach outperforms the approach of Garcia et al. [6].

In [14], we presented a tabu search method for solving the MCTOPTW problem. Oppositely to the approach of Garcia et al. [6] that tackles the same problem, we enforced an additional constraint to the problem that forbids any waiting time between consecutive visits. The method was shown to be competitive to the state of the art algorithms for solving the MCTOPTW problem.

Further, in [15], we extended MCTOPTW problem to MC Multiple TOPTW (MCMTOPTW) to model the trip itinerary for a group of tourists. Then, we solved the problem by devising an algorithm that is built on top of the approach presented in [14], which is used for construction of the starting solution. The proposed algorithm was again implemented under the framework of tabu search meta heuristic with a neighborhood structure that relays on three operators, namely *Separate* (a tourist from the group), *Join* (two tourists from two different subgroups) and *Insert* (a new POI into the trip of a tourist). The solution that is returned by the algorithm creates personalized itineraries for each tourist in the group, by allowing them to switch the company with different tourists during the trip, in accordance to their preferences.

III. Mathematical Modeling

We suppose that there are *N* POIs available in a city or region. Each POI *i* is characterized with a set of attributes, such as: coordinates (x_i, y_i), visit duration t_i, tourist satisfaction factor S_i, opening time o_i, closing time c_i and entry fee e_i. In addition, we suppose that traveling time t_{ij} between POI *i* and POI *j* is symmetric and it is known for all *m=1, ..., N* and *j=1, ..., N*, given that $i \neq j$. On the other hand, the tourists trip details, include: number of tours *M*, budget for the complete trip (all tours) B_{max} and duration of tour T_m, where *i=1, ..., M*. In general, individual tours can have different starting and ending times. Further, we assume that for each tour *m*, POI *1* and POI *N* are the starting and ending points, respectively. In practice, the starting and ending point could be the same, where in most cases that is a hotel (accommodation).

Next, based on [16], we formulate the envisioned trip planning problem as an integer programing problem, where we make use of two decision variables, namely x_{ijm} and y_{im} that are defined as in following. In addition, the starting time of a visit to POI *i* in tour *m* is denoted as s_{im}.

$$x_{ijm} = \begin{cases} 1, & \text{if visited point } i \text{ is followed with a visit to point } j \text{ in tour } m \\ 0, & \text{otherwise} \end{cases}$$

$$y_{im} = \begin{cases} 1, & \text{if point } i \text{ is visited in tour } m \\ 0, & \text{otherwise} \end{cases}$$

$$Max \sum_{m=1}^{M} \sum_{i=2}^{N-1} S_i y_{im}, \tag{1}$$

$$Min \sum_{m=1}^{M} \sum_{i=1}^{N} \sum_{\substack{j=1 \\ j \neq i}}^{N} t_{ij} x_{ijm}, \tag{2}$$

$$\sum_{m=1}^{M} \sum_{i=2}^{N-1} e_i y_{im} \leq B_{max}, \tag{3}$$

$$\sum_{m=1}^{M} \sum_{i=2}^{N-1} y_{im} \leq 1, \tag{4}$$

$$(y_{im} = 1) \implies (o_i \leq s_{im} \wedge s_{im} + t_i \leq c_i), \quad \forall i = 2, ..., N-1, \ \forall m = 1, ..., M \tag{5}$$

$$\sum_{i=1}^{N-1} \left(t_i y_{im} + \sum_{j=2}^{N} t_{ij} x_{ijm} \right) \leq T_m; \ \forall m = 1, ..., M \tag{6}$$

The first two constraints take part in a two component objective function for the envisioned problem, where Constraint (1) aims at maximizing the total collected score for tourist (by visiting POIs with the highest satisfaction factor), whereas Constraint (2) aims at reducing the total traveling time for the tourist throughout all tours. Constraint (3) limits the overall cost of all tours to the specified maximum budget B_{max}, whereas Constraint (4) makes sure that no POI is visited more then once. Constraint (5) ensures that the visits to individual POIs occur only during their respective opening hours. Finally, Constraint (6) limits the duration of individual tours to the respective maximal allowed durations.

The envisioned problem deviates from the original TOPTW problem [5] in two aspects: (1) it considers the entry fee of POIs by limiting the maximum allowed budget to a specified limit, and (2) it optimizes the solution not only based on the overall tourist satisfaction factor, but also based on the overall tourist traveling time.

IV. Solution Approach

The Simulated Annealing (SA) is a metaheuristic that is based on theories from thermodynamics where creation of the structure of crystals depends merely on the cooling function, which should neither be so quick or so slow, as defects in the crystal might appear. Analogical, in the field of optimization, SA uses a cooling function to determine whether the algorithm should also be able to select random solutions that might be worse then the current solution, or it should only accept better solutions, and thus act more as a hill climber. This feature enables SA algorithm to avoid premature convolution, thus it is able to escape from getting stuck in local optimum. The SA algorithm was initially presented by Kirkpatrick et al. in [17]. Recently, the SA algorithm was used by Lin & Yu [18] to successfully solve the related problem of TOPTW.

A. Solution Representation

The representation of a given current solution (denoted as S_c) is done by using an array of M members, where each member is a list, which itself contains the sequence of POIs that are part of a single tour (day). In addition, the POIs that are not included in any of the tours, are kept in separate list called "QueueList". Considering an example of a tourist visiting a city that has $N = 10$ POIs for a duration of $M = 2$ tours, an arbitrary candidate solution might be represented as in following:

$$S_c = \{[1, 8, 2, 5], [9, 7, 10]\}$$
$$QueueList = [3, 4, 6]$$

B. Neighborhood Exploration

There is a number of operators in the literature that could be used to solve the proposed problem. The *insert* operator [5] is used for insertion of new POIs in the itinerary, or *remove* operator to remove one or more POIs from the itinerary. In addition, the *swap/exchange* operator can be used to exchange POIs inside the itinerary with POIs outside of the itinerary, or also it can be applied to exchange POIs within itinerary itself (cross-exchange) [19].

In this implementation, the structure of the neighborhood exploration procedure consists of three operators, namely *insert*, *swap* and *shake*. The *swap* operator is only implemented to exchange POIs in the itinerary with POIs outside of the itinerary. Nevertheless, the cross exchange between the POIs within the itinerary occurs indirectly through *swap* operator, since when a POI goes outside of the itinerary at a given iteration, it has the chance of going back to the itinerary at a later iteration, and there exist probability that the same POI would go to some other position in the tour than it has been at the time when it was lastly removed.

C. Evaluation Function

The evaluation (fitness) of the solutions to the problem at hand are done by using a function that consists of two components, as given by Equation (7). The first component E_s represent the overall normalized satisfaction factor of the tourist with the POIs included into the itinerary, whereas the second component E_t represents the overall normalized satisfaction of the tourist with the overall traveling time during the tours. Both components are multiplied with their corresponding wight coefficients w_s and w_t, respectively, which are used to make a trade off between finding itineraries that include POIs with high satisfaction factors, or in finding itineraries with less travel time. In this implementation, the values of the weight coefficients are assumed to be set by the user, whereas in the future work, we aim to automatize them by taking a multi-objective approach, such as for instance the method of *Pareto front* of the space of candidate solutions.

In reference to Constraint (1) and Constraint (2), note that in order to have a maximizing fitness function, where both its components would have a maximization nature, the second component of Equation (7) needs to consider the complementary value of the normalized traveling time, since its original aim is to minimize the traveling time.

In order to make sure that individual components of the evaluation function do not predominate each other, their respective values need to be normalized based on their maximal possible values. In regard to the component of satisfaction factor with POIs, the normalization (Equation (9) is made by supposing a maximum number of POIs (denoted as τ) that can be visited during a tour (day), and by considering the satisfaction value of the POI with the highest satisfaction factor (denoted as ω). On the other hand, the normalization of the traveling time component is done by considering the total duration of all tours.

$$E(S) = w_s E_{s_{norm}} + w_t(1 - E_{t_{norm}}) \tag{7}$$

$$E_{s_{norm}} = \frac{\sum_{m=1}^{M} \sum_{i=2}^{N-1} S_i y_{im}}{M * \tau * \omega} \tag{8}$$

$$E_{t_{norm}} = \frac{\sum_{m=1}^{M} \sum_{i=1}^{N-1} \left(t_i y_{im} + \sum_{j=2}^{N} t_{ij} x_{ijm} \right)}{\sum_{m=1}^{M} T_m} \tag{9}$$

TABLE I. ALGORITHM PARAMETERS

Symbol	Description
α	Specifies the proportion of temperature decrease
β	Maximal temperature used to commence the search process
γ	Minimal temperature used to end the search process
δ	Maximal number of iterations without improvement
ϵ	Number of POIs to remove during the Shake step
ζ	Specifies one of the modes for generation of the initial solution

D. Simulated Annealing Implementation

The proposed SA algorithm can be fine tuned by using six different parameters. Table I gives more details about the utilized parameters, where the first column presents the respective symbols, whereas the second column explains the purpose for using the envisioned parameters.

As shown in the pseudo code (Figure 1), the parameters can be used to fine tune different mechanism within the algorithm. The ζ parameter sets one of five possible modes for generation of the initial solution, as described in following:

- *Mode 1* - inserts POIs into the itinerary in a random order
- *Mode 2* - initially orders POIs based on their satisfaction factor and then inserts them into the itinerary in a descending order, where POIs with highest satisfaction are inserted first
- *Mode 3* - orders POIs as *Mode 2*, except that it inserts them into the itinerary in a ascending order, where POIs with lowest satisfaction are considered first
- *Mode 4* - calculates the average distance of each POI with all other POIs, and then inserts them into the itinerary by giving priority to the POIs that have shorter average distance to the others
- *Mode 5* - inserts POIs into the itinerary by giving priority to POIs that have a shorter visit duration

The evaluation function described earlier (Subsection IV-C) is used to evaluate the candidate solutions during the search process, such as the initial solution at the start, and the neighbor solutions later. The iterative part of the algorithm depends on its main loop, which is controlled by the initial temperature, which, at the start, is set to its maximal value (parameter β), the temperature decreasing ratio (parameter α), the minimal temperature (parameter γ), and the cooling function. In this implementation, the cooling function follows a decreasing exponential law, which depends on current iteration t and parameter α. At a given iteration t, the temperature T_t is calculated by using Equation 10, where T_{t-1} represent the temperature in previous iteration.

$$T_t = T_{t-1} e^{-\alpha * t} \tag{10}$$

In a given iteration t, the neighborhood of a current solution consists of all neighbors (candidate solutions) that yield as results of trying to swap every POI p that is off the itinerary with every POI q that is currently in the itinerary. There are two mechanisms that are applied priory to evaluating the fitness of the neighbors. The first is elimination of non feasible

```
1:  procedure GENERATE ITINERARY(α, β, γ, δ, ϵ, ζ)
2:      S_c ← GenerateInitialSolution(ζ)
3:      Evaluate S_c
4:      S_b ← S_c
5:      T ← β
6:      t ← 0
7:      i ← 0
8:      while T > γ do
9:          for each POI p in QueueList do
10:             for each POI q in S_c do
11:                 S_n ← ApplySwap(p, q)
12:                 if S_n is a feasible solution then
13:                     Fill empty space in S_n
14:                     Evaluate S_n
15:                     if E(S_n) > E(S_c) then
16:                         S_c ← S_n
17:                         if E(S_c) > E(S_b) then
18:                             S_b ← S_c
19:                             i ← 0
20:                         else
21:                             i ← +1
22:                         end if
23:                     else
24:                         a ← GenerateRandomNumber(0, 1)
25:                         b ← (T−γ)/(β−γ) e^((E(S_n)−E(S_c))/T)
26:                         if a < b then
27:                             S_c ← S_n
28:                         end if
29:                     end if
30:                 end if
31:                 if i > δ then
32:                     S_c ← ApplyShake(ϵ)
33:                     Reorder POIs in S_c
34:                     Fill empty space in S_c
35:                 end if
36:             end for
37:         end for
38:         t ← +1
39:         T ← ApplyCoolingFunction(t, α)
40:     end while
41:     return S_b
42: end procedure
```

Fig. 1. The SA algorithm for generation of trip itinerary

neighbors, which do no meet the hard constraints, such as maximum budget, at most one visit to a POI and limited duration of the tours, whereas the second mechanism is about filling empty spaces in the itinerary that might have come out due to application of the envisioned operators. Depending on the size of the empty space, additional non included POIs are inserted into the itinerary.

This implementation follows the general guidelines of SA algorithm in reference to adaption of a neighbor solution as a next current solution. Therefore, a neighbor is always accepted as a next current solution when it is better then the actual current solution. Nevertheless, if a neighbor is worse then the actual current solution, it can still be accepted, with a certain probability, as next current solution. The probability of acceptance is higher if the corresponding quality of the

neighbor is not that worse then the quality of actual current solution, and if the running temperature is higher (this is usually the case in the early iterations of the SA algorithm when it undergoes its exploitative phase). In the pseudocode, variable a gets a random number between 0 and 1, whereas variable b also gets a value between 0 and 1 through the expression in Line 25. If the value of a happens to be smaller then the value of b, then the neighbor solution would be accepted as the next current solution.

If the algorithm undergoes a consecutive number of iterations without improvement (as defined by parameter δ) then the *shake* operator will be applied. Considering the ϵ parameter, the *shake* operator removes a number of POIs from the itinerary. Afterwards, at first, the POIs are reordered by being shifted as much as possible towards the beginning of the tours, so that the empty spaces are gathered at the end of the tours, then the mechanism for filling up the empty spaces (described above) is put into function, and lastly the modified solution is adapted as next current solution. The application of *shake* operator might be important as it might help the SA algorithm to escape from getting caught into the local optimum.

V. Computational Experiments

The algorithm is developed by using Java 1.7. The experiments are conducted by using a machine with an Intel Core Duo 2.5 GHz processor with a Windows 7 operating system and 3GB of RAM memory. In the following subsection, we describe the test set, and then in Subsection V-B, we present the experimental results.

A. Test Set

The test set is made of fifteen instances, where eleven of them consist of two tours (days), while the remaining four have 1, 3, 4 and 5 tours, respectively. In addition, the tourist interest about types (e.g. monument, park, castle, etc.) and categories (e.g. archeology, architecture, religious art, etc.) of POIs is modeled by using the mechanism of random number generation of Java programming language. Further, all fifteen instances of the test set are executed against fifty real POIs in city of Prishtina in Kosova. Each POI, besides its name and description, is also characterized with its estimated tourist satisfaction/interest factor, coordinates, opening and closing time, visit duration and entry fee. In addition, each of the fifteen instances is associated with an additional point (e.g. hotel, bus station, city down town, etc.), which serves, both as a starting and ending point for the tours. The distance between individual POIs is expressed in the unit of minutes.

The first ten instances, except for the tourist satisfaction factor, are characterized with same attribute values, as it follows: 2 tours, each tour starts at 08:00 hrs and ends at 17:00 hrs, break time 30 minutes, wight for satisfaction factor 60% and weight for travel time 40%. The other five instances have diverse attribute values.

B. Experimental results

The values of the algorithm parameters (except ζ) have been fine tuned through some preliminary experiments that have been conducted by using the complete test set. The fine tuned values are: $\alpha = 0.005, \beta = 4000, \gamma = 80, \delta = 210, \epsilon = 6$.

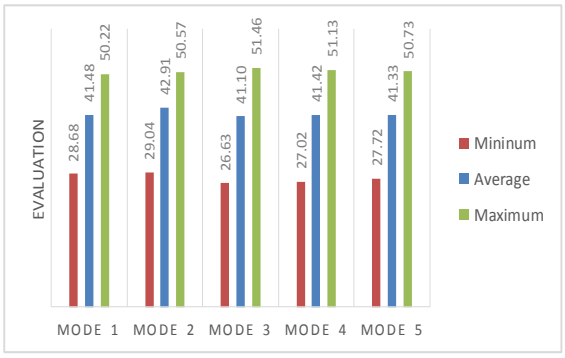

Fig. 2. Algorithm performance for different modes of initial solution

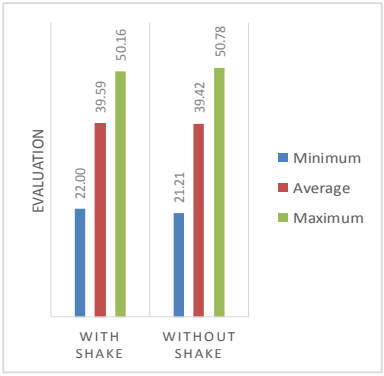

Fig. 3. Algorithm performance when *shake* operator is applied

In the following, we present the results about four types of experiments, such as: the performance of different modes of initial solution, the effect of *shake* operator into the search process, the variations between different executions of a given instance, and the time needed to execute instances with different number of tours. Each experiment includes ten executions for each instance of the test set. Therefore, the results presented below are averaged over the ten executions of individual instances.

Figure 2 shows that, in average, *Mode 2* of initial solution, where POIs with higher satisfaction factor are more common, performs better than the other considered modes. This result can be explained based on the indication that *Mode 2* of initial solution is more diverse, since most of its POIs have high satisfaction factor, hence they get more prone in being subject of exploration operators, and therefore make the algorithm more exploratory.

The diagram in Figure 3 depicts the averaged results over the complete test set, where the performance of the algorithm is presented in variance to the application of *shake* operator. By considering the average results, it can be noted down that the *shake* operator helps the algorithm to achieve only a slight improvement in the overall quality of solutions.

Figure 4 presents the variance in time over ten different executions of *Instance 12*, which consists of two tours. In terms of quality of solutions, for this particular instance, it is evident

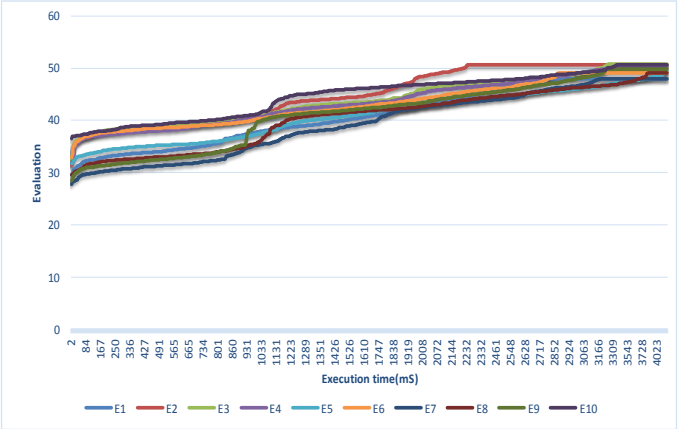

Fig. 4. Variations between different executions for instance *Instance 12*

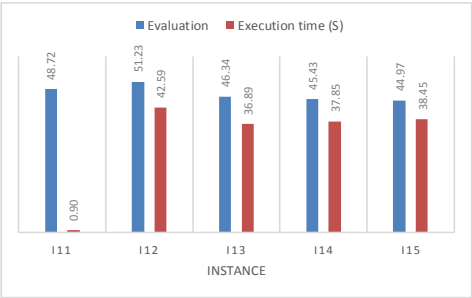

Fig. 5. Computation time for different number of tours

that the difference between different executions is at most a little less than 3 points, whereas the computation time, in the worst case scenario is a little less than 4 seconds. In addition, it can be noted down that, although, in some executions, the starting solutions are much worse, at the end of executions the algorithm manages to produce solutions with comparable quality to the cases where initial solution is better.

Figure 5 shows the performance of the algorithm when executed for instances 11 until 15, which consist of 1 to 5 tours, respectively. In terms of evaluation, the algorithm produces comparable results for all five considered instances, whereas the computation time for *Instance 11* (with one tour) is a little less than 1 second, and for other instances it is much higher.

The particular trip planning problem tackled in this paper differs from the related problems in the literature, therefore the results of the proposed algorithm can not be experimentally compared with the results of the state of the art algorithms.

VI. Conclusions

In this paper, we presented a new approach based on simulated annealing for planning the tourist trip itinerary. The computation time for itineraries with one tour is quite short, whereas for itineraries with two tours and more the average computation time is about 39 seconds. Although, the algorithm works slightly better by using *Mode 2* of initial solution (where POIs with higher satisfaction are given priority for insertion), we can conclude that the difference by using other modes is quite small. Despite our expectations, the application of *shake* operator does not prove to be very successful in improving the quality of solutions. Therefor, it remains an open question to investigate application of other operators that might be helpful in further improvement of the results.

References

[1] W. Souffriau, P. Vansteenwegen, J. Vertommen, G. V. Berghe, and D. V. Oudheusden, "A personalized tourist trip design algorithm for mobile tourist guides," *Applied Artificial Intelligence*, vol. 22, no. 10, pp. 964–985, 2008.

[2] T. Tsiligirides, "Heuristic methods applied to orienteering," *Journal of the Operational Research Society*, pp. 797–809, 1984.

[3] I. Chao, B. L. Golden, E. A. Wasil *et al.*, "The team orienteering problem," *European journal of operational research*, vol. 88, no. 3, pp. 464–474, 1996.

[4] M. G. Kantor and M. B. Rosenwein, "The orienteering problem with time windows," *Journal of the Operational Research Society*, pp. 629–635, 1992.

[5] P. Vansteenwegen, W. Souffriau, G. Vanden Berghe, and D. Van Oudheusden, "Iterated local search for the team orienteering problem with time windows," *Computers & Operations Research*, vol. 36, no. 12, pp. 3281–3290, 2009.

[6] A. Garcia, P. Vansteenwegen, W. Souffriau, O. Arbelaitz, and M. Linaza, "Solving multi constrained team orienteering problems to generate tourist routes," *status: published*, 2009.

[7] A. Divsalar, P. Vansteenwegen, and D. Cattrysse, "A variable neighborhood search method for the orienteering problem with hotel selection," *International Journal of Production Economics*, 2013.

[8] B. L. Golden, L. Levy, and R. Vohra, "The orienteering problem," *Naval Research Logistics (NRL)*, vol. 34, no. 3, pp. 307–318, 1987.

[9] P. Vansteenwegen, W. Souffriau, and D. V. Oudheusden, "The orienteering problem: A survey," *European Journal of Operational Research*, vol. 209, no. 1, pp. 1–10, 2011.

[10] W. Souffriau, J. Maervoet, P. Vansteenwegen, G. V. Berghe, and D. Van Oudheusden, "A mobile tourist decision support system for small footprint devices," in *Bio-Inspired Systems: Computational and Ambient Intelligence*. Springer, 2009, pp. 1248–1255.

[11] D. Gavalas, C. Konstantopoulos, K. Mastakas, G. Pantziou, and Y. Tasoulas, "Cluster-based heuristics for the team orienteering problem with time windows," in *Experimental Algorithms*. Springer, 2013, pp. 390–401.

[12] A. Garcia, O. Arbelaitz, P. Vansteenwegen, W. Souffriau, and M. T. Linaza, "Hybrid approach for the public transportation time dependent orienteering problem with time windows," in *Hybrid Artificial Intelligence Systems*. Springer, 2010, pp. 151–158.

[13] W. Souffriau, P. Vansteenwegen, G. Vanden Berghe, and D. Van Oudheusden, "The multi-constraint team orienteering problem with multiple time windows," *Transportation Science, Articles in Advance*, pp. 1–11, 2011.

[14] K. Sylejmani, J. Dorn, and N. Musliu, "A tabu search approach for multi constrained team orienteering problem and its application in touristic trip planning," in *Hybrid Intelligent Systems (HIS), 2012 12th International Conference on*. IEEE, 2012, pp. 300–305.

[15] K. Sylejmani, "Optimizing trip itinerary for tourist groups," Ph.D. dissertation, Vienna University of Technology, Faculty of Informatics, Austria, 2013.

[16] K. Sylejmani and A. Dika, "Solving touristic trip planning problem by using taboo search approach," *International Journal of Computer Science Issues(IJCSI)*, vol. 8, no. 5, 2011.

[17] S. Kirkpatrick, D. G. Jr., and M. P. Vecchi, "Optimization by simmulated annealing," *science*, vol. 220, no. 4598, pp. 671–680, 1983.

[18] S. W. Lin and V. F. Yu, "A simulated annealing heuristic for the team orienteering problem with time windows," *European Journal of Operational Research*, vol. 217, no. 1, pp. 94–107, 2012.

[19] F. Tricoire, M. Romauch, K. F. Doerner, and R. F. Hartl, "Heuristics for the multi-period orienteering problem with multiple time windows," *Computers & Operations Research*, vol. 37, no. 2, pp. 351–367, 2010.